**DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING**

**COLLEGE OF E&ME, NUST, RAWALPINDI**


# Software Bug & Project Tracking System

## EC 240: Database Engineering

## Semester Project Report

**SUBMITTED TO:**

Dr Shehzad

## SUBMITTED BY:

| Sr No. | Name | Reg No. | Role |
|---|---|---|---|
| 1 | Aleeza Rizwan (Group Leader) | 456143 | The Architect (ER Design, Schema, Report) |
| 2 | M Danyal Ejaz | 482858 | The Analyst (Data Seeding, SQL Queries) |
| 3 | M Shaheer Afzal | 481560 | The Developer (FastAPI Backend, Frontend UI) |
| 4 | M Ibrahim Abdullah | 454188 | The Data Engineer (DDL, Indexes, Views, Triggers) |

**DE-45 CE Syndicate A**

Submission Date: 8th May, 2026

# Table of Contents

# 1. Problem Description

## 1.1 Background and Motivation

Modern software development organizations generate large volumes of operational data: bug reports, feature requests, sprint timelines, developer assignments, and audit trails. Without a structured database system, this data becomes fragmented across spreadsheets, email threads, and informal channels, leading to duplicated effort, missed deadlines, and an inability to derive meaningful insights about team performance or product quality.

This project addresses that gap by designing and implementing a relational database-backed application, a Software Bug and Project Tracking System, that mirrors the core functionality of industry tools such as Atlassian Jira and GitHub Issues. The domain was selected because it is both technically rich and immediately familiar to software engineering students, making it possible to model realistic business rules with high precision.

## 1.2 Problem Statement

Software teams lack a centralized, structured system for tracking issues across projects and sprints, resulting in:

- Inability to identify which developers are overloaded or underperforming.
- No enforced workflow, bugs can be marked 'Closed' without ever being reviewed.
- Loss of accountability: no record of who changed a bug's status or when.
- Redundant data entry when the same issue is tracked in multiple places.
- No systematic reporting on sprint velocity, bug severity trends, or resolution time.

The proposed system resolves these issues through a rigorously normalized relational schema, enforced business rules via database triggers, analytical views, and a REST API front-end that exposes the data to end users.

## 1.3 Objectives

1. Design a conceptual ER model that captures the full complexity of the bug-tracking domain, including ISA specialization, weak entities, and multi-valued attributes.
2. Transform the ER model into a 3NF relational schema with appropriate primary keys, foreign keys, and CHECK constraints.
3. Implement the schema in Microsoft SQL Server (T-SQL) with 16 tables, 10 indexes, 5 views, and 5 triggers.
4. Seed the database with realistic data: 15 users, 5 projects, 5 sprints, and 55+ issues.
5. Write a minimum of 15 analytical SQL queries covering joins, subqueries, GROUP BY / HAVING, and complex conditions.
6. Build a FastAPI (Python) backend with pyodbc that exposes REST endpoints consuming the database.

### 1.4 Scope

The system covers five functional areas: user and role management, project and sprint management, issue lifecycle tracking (bugs, features, tasks), commenting and attachments, and analytical reporting. Authentication and multi-tenancy are out of scope for this academic prototype.

## 2. Literature Review

### 2.1 Relational Database Design Principles

The relational model, introduced by E.F. Codd (1970), organises data into tables where each row represents a unique fact and columns represent attributes. The Structured Query Language (SQL) provides a declarative interface for querying and manipulating relational data and has become the dominant standard for relational systems (Date, 2003).

Normalization theory, developed through the work of Codd, Boyce, and Fagin, provides a systematic framework for eliminating data redundancy and update anomalies. First Normal Form (1NF) requires atomicity; Second Normal Form (2NF) eliminates partial dependencies on composite keys; Third Normal Form (3NF) removes transitive dependencies. Achieving 3NF is considered sufficient for most operational systems, balancing data integrity with query performance (Elmasri & Navathe, 2016). This project normalizes all tables to 3NF.

### 2.2 Entity–Relationship Modelling

Chen (1976) introduced the Entity–Relationship (ER) model as a tool for conceptual database design. Extensions by Teorey et al. (1986) and Smith & Smith (1977) added specialization and generalization hierarchies, enabling object-oriented-style inheritance in relational modelling. This project uses the ISA hierarchy to model the three subtypes of Issue (Bug, Feature, Task), a pattern widely discussed in Elmasri & Navathe (2016) and implemented as a shared-primary-key strategy in the relational schema.

Weak entities, which require an identifying owner entity for their primary key, are also a foundational ER concept (Elmasri & Navathe, 2016). The COMMENT entity in this project is a canonical weak entity: a comment is meaningless without its parent issue, and its partial key (CommentNo) is only unique within the scope of a single IssueID.

### 2.3 SQL Enhancements: Triggers, Views, and Indexes

Triggers are database objects that execute procedural logic automatically in response to data modification events (INSERT, UPDATE, DELETE). They are widely used to enforce complex business rules that cannot be expressed as simple CHECK constraints (Ramakrishnan & Gehrke, 2003). This project uses five triggers, including a state-machine enforcement trigger (TR_Issue_StatusTransition) and a subtype integrity trigger (TR_Issue_SubtypeEnforce).

Views provide a virtual table interface over underlying queries, improving abstraction, security, and query simplicity (Date, 2003). The five dashboard views in this project (e.g., VW_DEVELOPER_WORKLOAD, VW_SPRINT_VELOCITY) encapsulate complex multi-table joins, allowing the application layer to issue simple SELECT * FROM view_name calls.

Indexes are auxiliary data structures that accelerate query execution by reducing the number of rows scanned. Non-clustered indexes with INCLUDE columns, as used in this project, allow the query

engine to satisfy a query entirely from the index, avoiding a table lookup (Microsoft SQL Server documentation, 2024).

### 2.4 Web APIs and Database Integration

FastAPI (Ramírez, 2018) is a modern Python web framework that leverages Python type hints and Pydantic models for automatic request validation and OpenAPI documentation generation. Combined with pyodbc, the Python Database API (DB-API 2.0) driver for ODBC-compliant databases, it provides a lightweight, high-performance bridge between a REST client and a SQL Server backend. The use of dependency injection (the get_db_connection generator) follows best practices for connection lifecycle management, ensuring connections are closed after each request and preventing resource leaks.

### 2.5 Industry Precedents

The Jira platform (Atlassian, 2002) pioneered the concept of issue-based project tracking at scale. GitHub Issues (GitHub, 2011) popularized a lightweight variant tightly integrated with source control. Academic literature on software project management databases (Kerzner, 2017) confirms that the entities modelled in this project, projects, sprints, issues, assignments, and audit logs, represent the minimal set required for a functional agile tracking tool.

## 3. System Architecture

### 3.1 Three-Tier Architecture Overview

The system follows a classic three-tier architecture: a presentation tier (HTML/CSS frontend), a logic tier (FastAPI Python backend), and a data tier (Microsoft SQL Server: BugTrackerDB). Each tier communicates via well-defined interfaces: the frontend calls REST endpoints via HTTP JSON; the backend communicates with SQL Server via pyodbc using T-SQL statements.

| Tier | Technology | Responsibility |
|---|---|---|
| Presentation | HTML / CSS / Jinja2 | Dashboard, Issue List, Create Form, Detail Page, Reports |
| Logic | Python 3.11 + FastAPI | Input validation (Pydantic), endpoint routing, transaction management |
| Data | Microsoft SQL Server 2022 | 16 tables, 10 indexes, 5 views, 5 triggers, BugTrackerDB |

## 3.2 System Flow

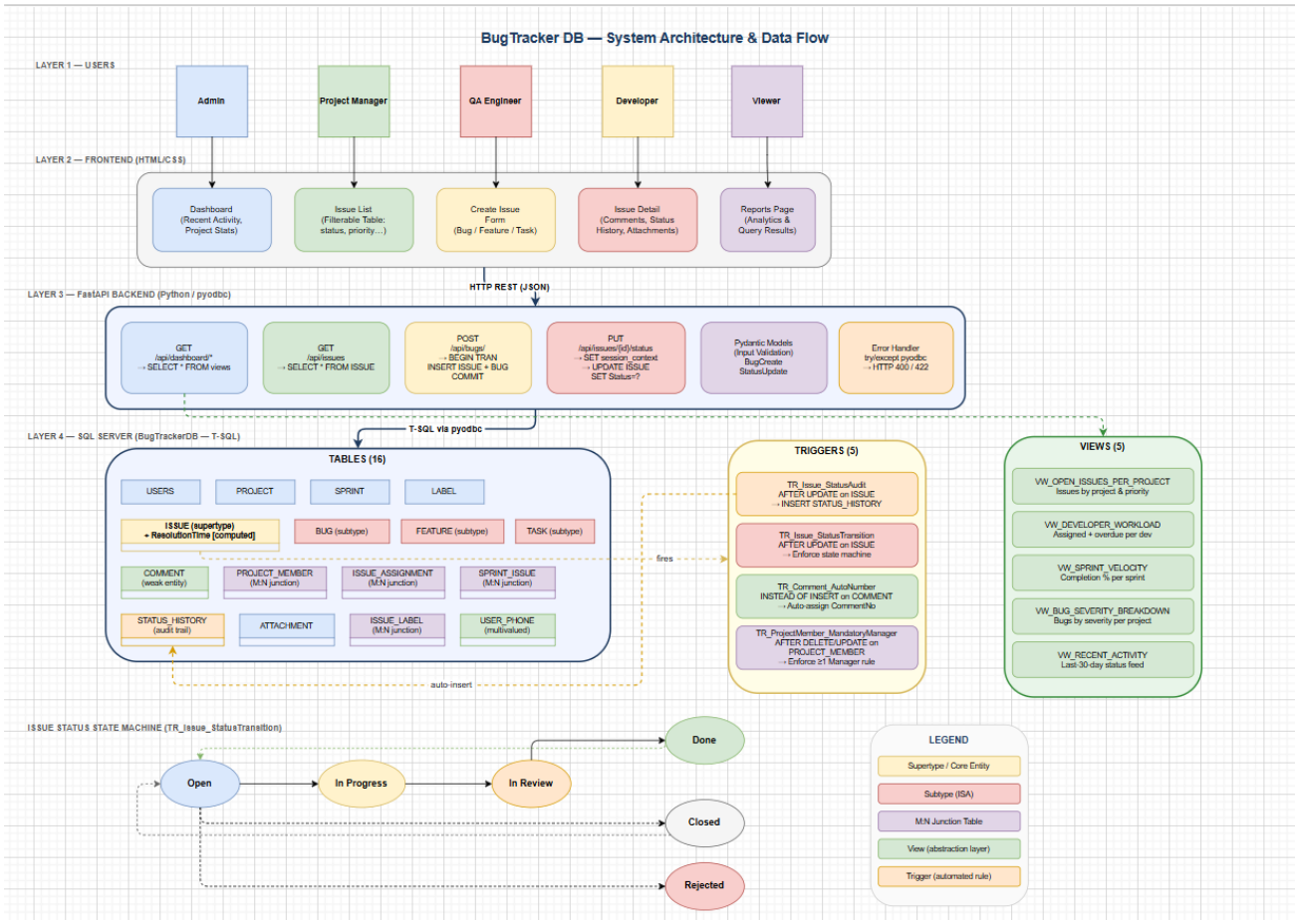


*Figure 1: System Architecture & Data Flow Diagram*

A request follows this path through the system:

7. User interacts with a page in the browser (e.g., submits the Create Issue form).
8. The frontend sends an HTTP request (POST /api/bugs/) to the FastAPI backend.
9. Pydantic validates the request payload; if invalid, a 422 Unprocessable Entity response is returned immediately.
10. The backend opens a pyodbc connection to BugTrackerDB.
11. For a bug creation, the backend executes a T-SQL transaction block inserting into ISSUE then BUG.
12. TR_Issue_SubtypeEnforce fires at statement end; if the BUG row is missing, the transaction is rolled back.
13. On success, conn.commit() is called. The response is serialized to JSON and returned to the browser.
14. The frontend re-fetches the issue list (GET /api/issues) and updates the UI table.

*Note: The flow diagram (BugTracker_SystemFlow.drawio) included with this submission illustrates all four layers, users, frontend screens, FastAPI endpoints, and the SQL Server database objects, and their interconnections. It should be opened in draw.io (diagrams.net) to view the full interactive diagram.*

### 3.3 FastAPI Endpoint Inventory

| Method | Endpoint | SQL Operation | Description |
|---|---|---|---|
| GET | /api/dashboard/recent-activity | SELECT * FROM VW_RECENT_ACTIVITY | Last 30 days of status changes |
| GET | /api/dashboard/developer-workload | SELECT * FROM VW_DEVELOPER_WORKLOAD | Workload per developer |
| GET | /api/dashboard/project-stats | SELECT * FROM VW_OPEN_ISSUES_PER_PROJECT | Open issues by priority per project |
| GET | /api/issues | SELECT * FROM ISSUE | Full issue list for the Issue List page |
| PUT | /api/issues/{id}/status | SET session_context → UPDATE ISSUE | Status update with audit trail injection |
| POST | /api/bugs/ | BEGIN TRAN → INSERT ISSUE + BUG → COMMIT | Transactional bug creation (subtype rule) |

### 3.4 Database Connection & Transaction Management

The get_db_connection function is a FastAPI dependency that yields a pyodbc connection and guarantees closure via a try/finally block. pyodbc operates in manual-commit mode by default (autocommit=False), meaning all writes are deferred until an explicit conn.commit() or reversed by conn.rollback(). This behaviour is exploited in the bug creation endpoint, where two INSERT statements must both succeed atomically.

For status updates, the session context mechanism (EXEC sys.sp_set_session_context @key=N'CurrentUserID') injects the logged-in user's ID into the SQL Server session memory, allowing the TR_Issue_StatusAudit trigger to record the correct actor in STATUS_HISTORY without requiring a separate application-level insert.

## 4. Requirements Analysis

### 4.1 Intended Users

| Role | Seeded Examples | Primary Capabilities |
|---|---|---|
| Admin | Aleeza (UserID 1) | Full system access; manage users and roles |
| Manager | Ibrahim, Shaheer, Talha | Create projects, manage sprints, assign members |
| Developer | Danyal, Eman, Furkan, Gulshan, Haidar, Bilal, Jameel | View assigned issues, update status, post comments |
| QA | Mahad, Noor, Osman | File bug reports, set severity, verify fixes |

| | | |
|---|---|---|
| Viewer | Ahmad | Read-only access to dashboards and reports |

### 4.2 Functional Requirements

15. Users can register with a role (Admin, Developer, QA, Manager, Viewer) and a unique email address.
16. Users can create new projects; each project must have at least one member with MemberRole = 'Manager'.
17. Project managers can add or remove members; the mandatory-manager trigger prevents removal of the last manager.
18. Users can create issues of type Bug, Feature, or Task, linked to a project and a reporter.
19. Each issue carries a priority (Low, Medium, High, Critical) and a status (Open → In Progress → In Review → Done / Closed / Rejected).
20. A Bug issue requires a matching row in the BUG table (Severity + optional StepsToReproduce).
21. Issues can be assigned to multiple developers (many-to-many via ISSUE_ASSIGNMENT).
22. Issues can be tagged with multiple labels (many-to-many via ISSUE_LABEL).
23. Project managers can create sprints with start/end dates and assign issues to them.
24. Developers can update issue status; illegal transitions are blocked by TR_Issue_StatusTransition.
25. Every status change is automatically recorded in STATUS_HISTORY by TR_Issue_StatusAudit.
26. Users can post comments on issues; CommentNo is auto-assigned per issue by TR_Comment_AutoNumber.
27. Users can attach files or URLs to issues.
28. The system exposes analytical dashboard views: workload, sprint velocity, bug severity breakdown, recent activity.
29. All database operations are accessible through the FastAPI REST interface and rendered in the HTML frontend.

### 4.3 Business Rules and Constraints

- **BR-01:** Each project must retain at least one member with MemberRole = 'Manager' at all times (enforced by TR_ProjectMember_MandatoryManager).
- **BR-02:** Issue status must follow the defined state machine: Open → In Progress → In Review → Done. Direct jumps to Closed or Rejected are allowed from Open. Re-open from Done or Closed to Open is allowed (enforced by TR_Issue_StatusTransition).
- **BR-03:** Every Bug-type Issue must have a corresponding row in the BUG table within the same transaction (enforced by TR_Issue_SubtypeEnforce).
- **BR-04:** A Comment cannot exist without its parent Issue (weak entity; ON DELETE CASCADE enforces this).
- **BR-05:** Issue Priority must be one of: Low, Medium, High, Critical (CHECK constraint).

- **BR-06:** Project EndDate, if provided, must be ≥ StartDate (CHECK constraint CK_Project_Dates).
- **BR-07:** Label Color, if provided, must be a valid 6-digit hex code in the format #RRGGBB (CHECK with LIKE pattern).
- **BR-08:** Feature BusinessValue, if provided, must be an integer between 1 and 10 (CHECK constraint).

# 5. Conceptual Design: ER Model

## 5.1 Entity Summary

The ER model contains 8 core entity types (plus 4 junction tables resolved from M:N relationships).

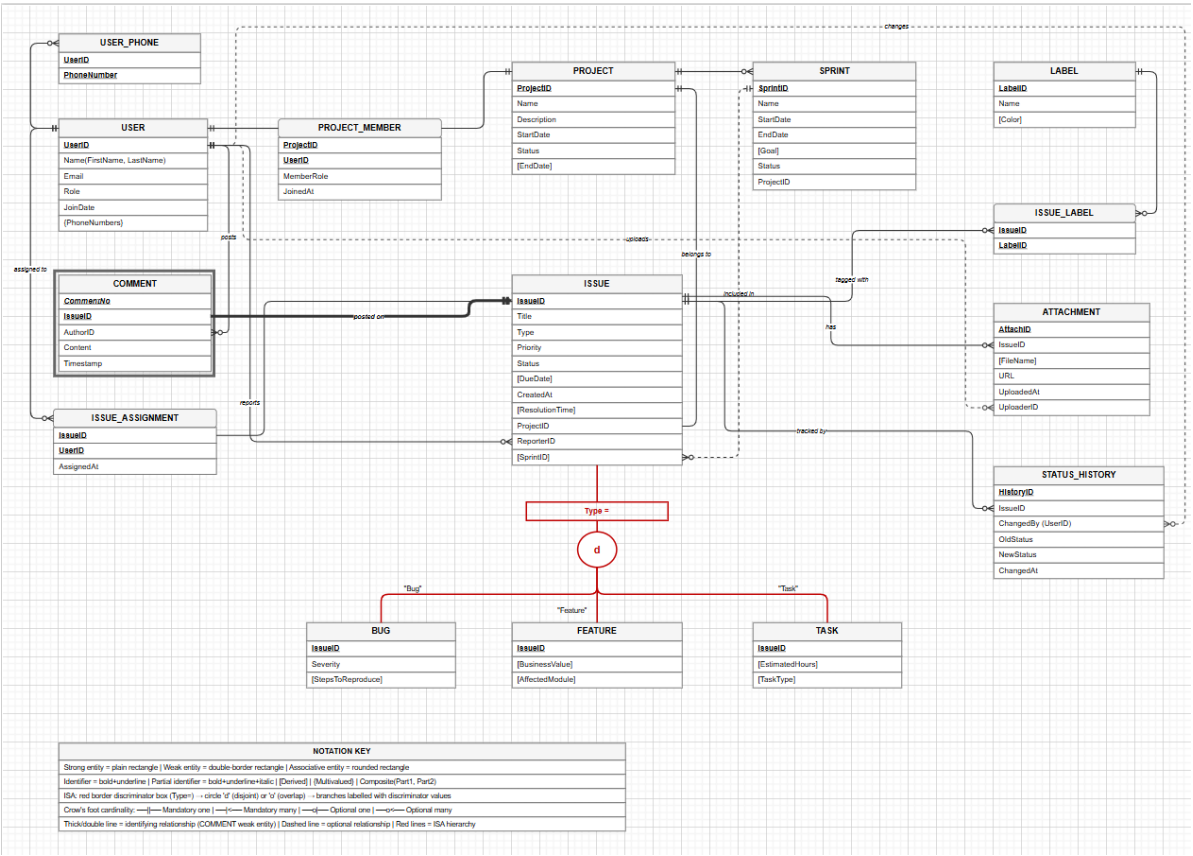


*Figure 2: Entity-Relationship Diagram*

*Note: The diagram (BugTracker_ERD.drawio) is also included with this submission.*

| Entity | Type | Primary Key | Notable Attributes |
|---|---|---|---|
| USERS | Strong | UserID | FirstName, LastName, Email (UNIQUE), Role (CHECK), JoinDate |
| USER_PHONE | Multi-valued | (UserID, PhoneNumber) | Resolves the multi-valued phone attribute of USERS |

| | | | |
|---|---|---|---|
| PROJECT | Strong | ProjectID | Name, StartDate, EndDate, Status (CHECK), Description |
| SPRINT | Strong | SprintID | Name, StartDate, EndDate, Goal, Status, ProjectID (FK) |
| LABEL | Strong | LabelID | Name (UNIQUE), Color (hex CHECK) |
| ISSUE | Supertype | IssueID | Title, Type, Priority, Status, DueDate, CreatedAt, ResolutionTime (computed/derived), ProjectID (FK), ReporterID (FK) |
| BUG | Subtype / ISA | IssueID (shared) | Severity (CHECK), StepsToReproduce |
| FEATURE | Subtype / ISA | IssueID (shared) | BusinessValue (1–10), AffectedModule |
| TASK | Subtype / ISA | IssueID (shared) | EstimatedHours, TaskType (CHECK) |
| COMMENT | Weak Entity | (CommentNo, IssueID) | AuthorID (FK), Content, Timestamp; CommentNo auto-assigned by trigger |
| ATTACHMENT | Strong | AttachID | IssueID (FK), FileName, URL, UploadedAt, UploaderID (FK) |
| STATUS_HISTORY | Strong | HistoryID (IDENTITY) | IssueID (FK), ChangedBy (FK), OldStatus, NewStatus, ChangedAt |

### 5.2 Relationships

| Relationship | Entities | Cardinality | Resolved By |
|---|---|---|---|
| works on | USERS — PROJECT | M:N | PROJECT_MEMBER junction (+ MemberRole, JoinedAt) |
| assigned to | USERS — ISSUE | M:N | ISSUE_ASSIGNMENT junction (+ AssignedAt) |
| tagged with | ISSUE — LABEL | M:N | ISSUE_LABEL junction |
| included in | ISSUE — SPRINT | M:N | SPRINT_ISSUE junction |
| belongs to | ISSUE — PROJECT | M:1 | FK_Issue_Project (NOT NULL) |
| reported by | ISSUE — USERS | M:1 | FK_Issue_Reporter (NOT NULL) |
| has comments | ISSUE — COMMENT | 1:N (total on COMMENT) | FK_Comment_Issue + weak entity PK |
| has attachments | ISSUE — ATTACHMENT | 1:N | FK_Attach_Issue |
| ISA | ISSUE — BUG/FEATURE/TASK | Specialisation | Shared PK; subtype enforced by TR_Issue_SubtypeEnforce |

### 5.3 Advanced ER Features

- **Specialization / Generalization:** The ISSUE entity is the supertype. BUG, FEATURE, and TASK are partial, overlapping subtypes (though in practice a given issue has exactly one subtype). The shared-primary-key strategy maps each subtype to its own table, sharing IssueID as both PK and FK to ISSUE. Subtype correctness is enforced by TR_Issue_SubtypeEnforce.

- **Weak Entity:** COMMENT is a weak entity identified by the composite key (CommentNo, IssueID). CommentNo is a partial key that is only meaningful within a specific issue. TR_Comment_AutoNumber auto-increments CommentNo per issue.
- **Multi-valued Attribute:** A user may have multiple phone numbers. USER_PHONE resolves this by storing one row per (UserID, PhoneNumber) pair, with a composite PK.
- **Derived Attribute:** ResolutionTime on ISSUE is a computed column: DATEDIFF(HOUR, CreatedAt, GETDATE()) for issues with status Done or Closed, NULL otherwise. It is never stored, always derived.
- **Composite Attribute:** UserName is conceptually composite (FirstName + LastName), stored as two separate VARCHAR columns per 1NF and concatenated in views as FirstName + ' ' + LastName.

## 6. Logical Design: Relational Schema

### 6.1 Schema Transformation Rules Applied

- Each strong entity type → one table with its attributes; primary key identified.
- Each weak entity → table with composite PK (partial key + owner PK); FK to owner with ON DELETE CASCADE.
- Each M:N relationship → junction table with composite PK from both participating PKs.
- Each 1:N relationship → FK placed in the 'many' side table.
- Specialization (shared PK) → subtype table shares PK with supertype and carries additional subtype-specific attributes.
- Multi-valued attribute → separate table with composite PK.
- Derived attribute → implemented as SQL Server computed column (not stored).

### 6.2 Full Relational Schema

The complete schema is depicted in the BugTracker_Relational_Schema.drawio file.

*Note: The diagram (BugTracker_Relational_Schema.drawio) is also included with this submission*

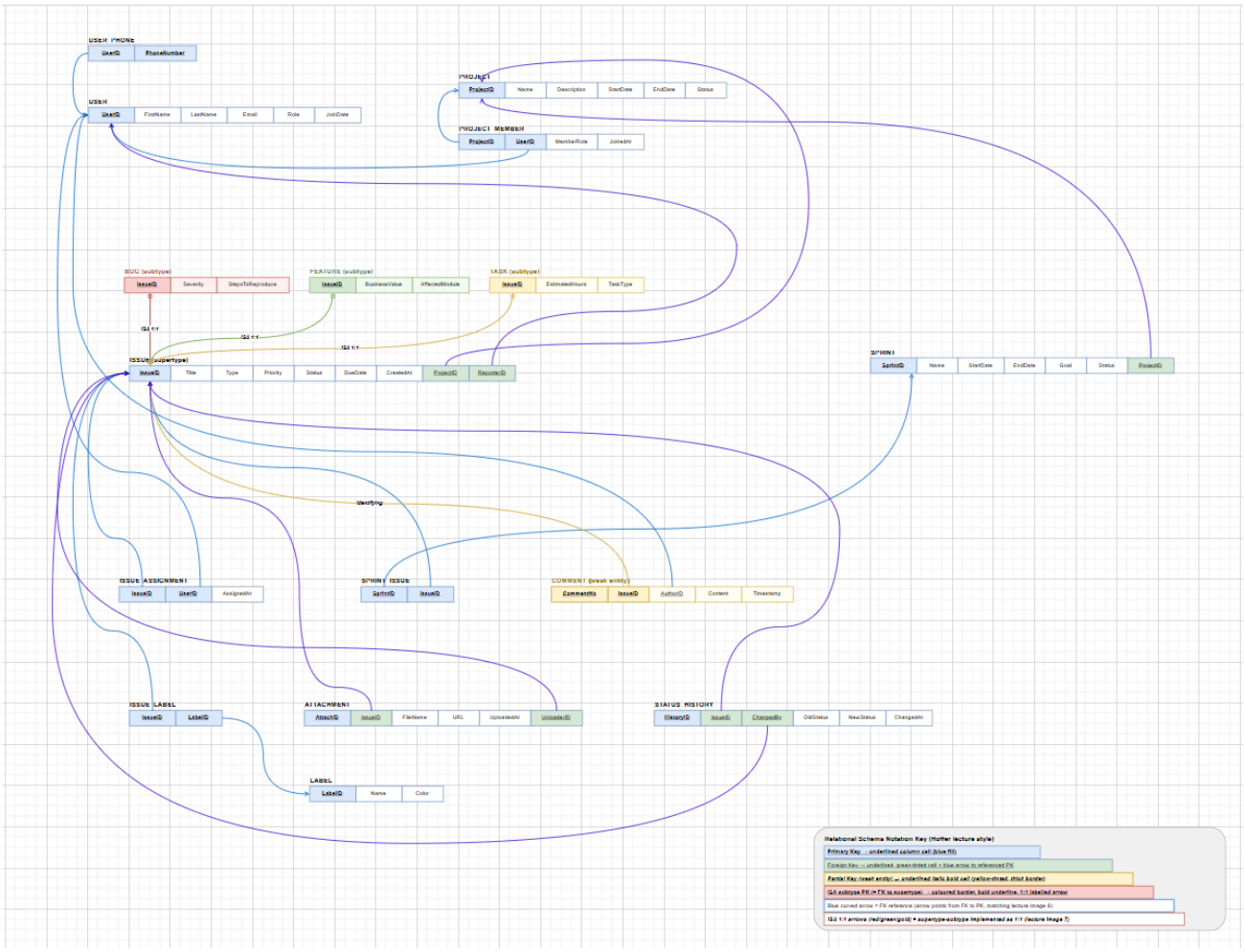

*Figure 3: Relational Schema*

It is also defined in bug_tracker_data_engineer.sql. A summary follows:

```
USERS         (UserID PK, FirstName NN, LastName NN, Email
UNIQUE NN, Role CHECK NN, JoinDate NN DEFAULT)
USER_PHONE    (UserID FK, PhoneNumber, PK(UserID,PhoneNumber))
PROJECT       (ProjectID PK, Name NN, Description, StartDate NN,
EndDate, Status CHECK NN, CHECK EndDate>=StartDate)
PROJECT_MEMBER (ProjectID FK, UserID FK, MemberRole CHECK NN,
JoinedAt, PK(ProjectID,UserID))
SPRINT        (SprintID PK, Name NN, StartDate NN, EndDate NN,
Goal, Status CHECK, ProjectID FK NN)
LABEL         (LabelID PK, Name UNIQUE NN, Color CHECK)
ISSUE         (IssueID PK, Title NN, Type CHECK NN, Priority
CHECK NN, Status CHECK NN,
               DueDate, CreatedAt DEFAULT, ResolutionTime
COMPUTED, ProjectID FK NN, ReporterID FK NN)
```

```
BUG          (IssueID PK FK→ISSUE, Severity CHECK NN,
StepsToReproduce)
FEATURE      (IssueID PK FK→ISSUE, BusinessValue CHECK,
AffectedModule)
TASK         (IssueID PK FK→ISSUE, EstimatedHours CHECK,
TaskType CHECK)
ISSUE_ASSIGNMENT (IssueID FK, UserID FK, AssignedAt DEFAULT,
PK(IssueID,UserID))
SPRINT_ISSUE    (SprintID FK, IssueID FK, PK(SprintID,IssueID))
COMMENT      (CommentNo, IssueID FK, PK(CommentNo,IssueID),
AuthorID FK NN, Content NN, Timestamp DEFAULT)
ISSUE_LABEL  (IssueID FK, LabelID FK, PK(IssueID,LabelID))
ATTACHMENT   (AttachID PK, IssueID FK NN, FileName, URL NN,
UploadedAt DEFAULT, UploaderID FK NN)
STATUS_HISTORY (HistoryID IDENTITY PK, IssueID FK NN, ChangedBy
FK NN, OldStatus, NewStatus NN, ChangedAt DEFAULT)
```

### 6.3 Normalization Verification

| Table | 1NF | 2NF | 3NF | Notes |
|---|---|---|---|---|
| USERS | ✓ | ✓ | ✓ | All attributes depend solely on UserID |
| PROJECT | ✓ | ✓ | ✓ | No transitive dependencies; EndDate check is a constraint, not a dependency |
| ISSUE | ✓ | ✓ | ✓ | ResolutionTime is computed, not a functional dependency on any non-key attribute |
| BUG / FEATURE / TASK | ✓ | ✓ | ✓ | Single-attribute PK (shared); subtype attributes depend entirely on IssueID |
| PROJECT_MEMBER | ✓ | ✓ | ✓ | Composite PK; MemberRole depends on the full key (ProjectID,UserID) |
| ISSUE_ASSIGNMENT | ✓ | ✓ | ✓ | AssignedAt depends on the full composite key |
| COMMENT | ✓ | ✓ | ✓ | Weak entity; all attributes depend on full composite PK (CommentNo, IssueID) |
| STATUS_HISTORY | ✓ | ✓ | ✓ | IDENTITY PK; all attributes are facts about a single audit event |

## 7. SQL Implementation

### 7.1 Data Definition Language (DDL)

All 16 tables are created in bug_tracker_data_engineer.sql in dependency order (parent tables before child tables). Each table definition includes primary key constraints, foreign key constraints with appropriate ON DELETE behaviour, and CHECK constraints enforcing domain rules.

Key DDL choices:

- **ON DELETE CASCADE:** Applied to child tables (BUG, COMMENT, ISSUE_ASSIGNMENT, SPRINT_ISSUE, etc.) so that deleting an ISSUE automatically cleans up all dependent records, preventing orphaned rows.
- **IDENTITY column:** STATUS_HISTORY.HistoryID uses IDENTITY(1,1), delegating key generation to SQL Server so that the application layer never needs to manage audit record IDs.
- **Computed column:** ISSUE.ResolutionTime is a non-persisted computed column. SQL Server evaluates it on each read, ensuring it always reflects the current state without requiring application-level logic.

### 7.2 Data Seeding

Danyal seeded the database with the following realistic sample data to satisfy rubric requirements:

| Table | Rows Inserted | Method |
|---|---|---|
| USERS | 15 | Explicit INSERT with named real-sounding data |
| PROJECT | 5 | Explicit INSERT: Alpha, Mobile App, Cloud Migration, External API, Internal CRM |
| SPRINT | 5 | One per project, varying statuses (Active, Completed, Planning) |
| ISSUE | 55 | WHILE loop; type distributed 1/3 Bug, 1/3 Feature, 1/3 Task |
| BUG / FEATURE / TASK | 55 (combined) | Inserted inside loop immediately after parent ISSUE |
| ISSUE_ASSIGNMENT | 55 | Every issue assigned to a rotating developer |
| SPRINT_ISSUE | ~22 | Issues for projects 1 and 2 added to sprints 1 and 2 |
| LABEL | 3 | Front-End, Back-End, Security |
| STATUS_HISTORY | ~25 (auto) | Generated automatically by TR_Issue_StatusAudit on UPDATE |

A key seeding detail: TR_Issue_SubtypeEnforce was temporarily disabled during the bulk WHILE loop insert (ALTER TABLE ISSUE DISABLE TRIGGER) to allow the two-statement pattern inside a loop without nesting explicit transactions. The trigger was re-enabled immediately after seeding, restoring business rule enforcement for all subsequent operations.

### 7.3 Analytical Queries

The following 15 queries were implemented by Danyal to satisfy the rubric requirement for joins, subqueries, aggregations, and complex conditions:

| # | Query Description | SQL Features Used |
|---|---|---|
| Q1 | All open critical bugs ordered by due date | WHERE, ORDER BY |

| | | |
|---|---|---|
| Q2 | Top 5 developers by resolved issues (last 30 days) | JOIN, WHERE DATEADD, GROUP BY, ORDER BY, TOP |
| Q3 | All overdue issues (DueDate < today AND status != closed) | WHERE with compound condition, GETDATE() |
| Q4 | Monthly issue creation trend | GROUP BY YEAR/MONTH, ORDER BY, COUNT |
| Q5 | Sprint completion rate (issues Done/Total per sprint) | SELECT from VW_SPRINT_VELOCITY |
| Q6 | Developers with >5 unresolved issues | JOIN, GROUP BY, HAVING COUNT > 5 |
| Q7 | Issues re-opened (status changed from Done/Closed → Open) | JOIN STATUS_HISTORY, WHERE OldStatus IN, NewStatus = 'Open' |
| Q8 | Average resolution time per project | JOIN, GROUP BY, AVG(ResolutionTime) |
| Q9 | Users who reported bugs but are never assigned to fix any | NOT EXISTS subquery |
| Q10 | Issues with no comments | NOT EXISTS subquery on COMMENT |
| Q11 | Most used labels across all projects | JOIN ISSUE_LABEL, GROUP BY, ORDER BY COUNT DESC |
| Q12 | Issues assigned to multiple developers simultaneously | JOIN ISSUE_ASSIGNMENT, GROUP BY, HAVING COUNT > 1 |
| Q13 | Full audit trail for a given issue | JOIN STATUS_HISTORY + USERS + ISSUE |
| Q14 | Projects with no currently active sprint | LEFT JOIN SPRINT, WHERE SprintID IS NULL OR Status != 'Active' |
| Q15 | Duplicate issue titles within the same project | Self-join on ISSUE WHERE i1.IssueID < i2.IssueID AND same ProjectID and Title |

## 7.4 Practical Enhancements

### 7.4.1 Indexes (10 implemented)

Ibrahim created 10 non-clustered indexes covering the most frequent query access patterns:

| Index Name | Table | Columns | Purpose |
|---|---|---|---|
| IX_Issue_Project_Status | ISSUE | (ProjectID, Status) INCLUDE(Title, Priority…) | Covers 'open issues in project' queries with a single index seek |
| IX_Issue_Priority | ISSUE | (Priority, Status) | Priority-based dashboard sorting (Critical first) |
| IX_Issue_Type_Status | ISSUE | (Type, Status) | Bug/Feature/Task count queries |
| IX_IssueAssignment_UserID | ISSUE_ASSIGNMENT | (UserID) INCLUDE(IssueID…) | Developer workload view (join on UserID) |
| IX_SprintIssue_SprintID | SPRINT_ISSUE | (SprintID) INCLUDE(IssueID) | Sprint velocity queries |
| IX_Sprint_Project_Status | SPRINT | (ProjectID, Status) | Per-project sprint listing |

| | | | |
|---|---|---|---|
| IX_StatusHistory_Issue_Time | STATUS_HISTORY | (IssueID, ChangedAt DESC) | Audit trail lookups ordered by time |
| IX_Comment_Issue_Time | COMMENT | (IssueID, Timestamp ASC) | Comments for an issue ordered chronologically |
| IX_Users_Email | USERS | (Email) | Login / email search |
| IX_PM_UserID_Role | PROJECT_MEMBER | (UserID, MemberRole) | 'Projects managed by user X' queries |

### 7.4.2 Views (5 implemented)

| View | Purpose | Used By |
|---|---|---|
| VW_OPEN_ISSUES_PER_PROJECT | Open issue count by priority per project | Project stats dashboard panel |
| VW_DEVELOPER_WORKLOAD | Total/active/overdue/resolved per developer | Developer workload page |
| VW_SPRINT_VELOCITY | Completion % per sprint | Sprint progress bar |
| VW_BUG_SEVERITY_BREAKDOWN | Bug count by severity and resolution status per project | QA dashboard |
| VW_RECENT_ACTIVITY | Last 30 days of status changes with actor names | Activity feed |

### 7.4.3 Triggers (5 implemented)

| Trigger | Event | Business Rule Enforced |
|---|---|---|
| TR_Issue_StatusAudit | AFTER UPDATE on ISSUE | Auto-inserts STATUS_HISTORY row on every status change; reads CurrentUserID from session context |
| TR_Issue_StatusTransition | AFTER UPDATE on ISSUE | Enforces legal state machine; RAISERROR and ROLLBACK on illegal transition |
| TR_Comment_AutoNumber | INSTEAD OF INSERT on COMMENT | Auto-assigns CommentNo as MAX(CommentNo)+1 scoped per IssueID |
| TR_ProjectMember_MandatoryManager | AFTER DELETE/UPDATE on PROJECT_MEMBER | Prevents removal of last manager; RAISERROR and ROLLBACK |
| TR_Issue_SubtypeEnforce | AFTER INSERT on ISSUE | Ensures each Bug/Feature/Task ISSUE has a matching subtype row within the same transaction |

# 8. Front-End Interface

## 8.1 Technology Stack

- Backend Framework: FastAPI (Python 3.11) provides REST endpoints with automatic OpenAPI documentation at /docs.
- Database Driver: pyodbc with ODBC Driver 17 for SQL Server; connection string targets BugTrackerDB on localhost\SQLEXPRESS.
- Frontend: HTML / CSS templates served via FastAPI's TemplateResponse; forms use standard HTML POST.
- Data Validation: Pydantic models (StatusUpdate, BugCreate) reject malformed payloads before they reach the database.

### 8.2 Screen Descriptions

| Screen | URL | SQL Executed | Description |
|---|---|---|---|
| Dashboard | / | SELECT * FROM VW_RECENT_ACTIVITY; VW_OPEN_ISSUES_PER_PROJECT | Summary panels: activity feed, project stats, developer workload |
| Developer Workload | /dashboard/developer-workload | SELECT * FROM VW_DEVELOPER_WORKLOAD | Table of developers with assigned, active, overdue, resolved counts |
| Issue List | /issues | SELECT * FROM ISSUE | All 55 seeded issues in a sortable, filterable table |
| Issue Detail | /issues/{id} | SELECT ISSUE + COMMENT JOIN USERS + STATUS_HISTORY | Full issue view with comment thread and audit trail |
| Create Issue (Bug) | /bugs/new | POST → BEGIN TRAN INSERT ISSUE+BUG COMMIT | Form for creating a new bug; validates subtype fields |
| Update Status | /issues/{id}/status (PUT) | SET session_context; UPDATE ISSUE SET Status=? | Inline status change with audit logging |
| Reports | /reports | Executes Queries Q1–Q5 on demand | Analytical report page with table output for 5 key queries |

### 8.3 SQL-to-UI Mapping

Every button or form submission in the frontend maps to a documented SQL operation. The key mappings are:

- **'Mark In Progress' button:** PUT /api/issues/{id}/status → EXEC sys.sp_set_session_context … → UPDATE ISSUE SET Status='In Progress'
- **'Submit New Bug' form:** POST /api/bugs/ → BEGIN TRAN; INSERT INTO ISSUE …; INSERT INTO BUG …; COMMIT
- **'Load Dashboard' (on page load):** GET /api/dashboard/recent-activity → SELECT * FROM VW_RECENT_ACTIVITY
- **'View Developer Workload' button:** GET /api/dashboard/developer-workload → SELECT * FROM VW_DEVELOPER_WORKLOAD

- **'Run Reports' button:** Direct SQL calls to Q1–Q5 analytical queries; results rendered in labelled HTML tables

## 9. Results

### 9.1 Database Verification Output

After running the full SQL script in SSMS, the verification queries at the end of bug_tracker_data_engineer.sql produced the following outputs (screenshots from the demo recording):

| Verification Query | Expected Result | Status |
|---|---|---|
| SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE='BASE TABLE' | 16 tables listed | ✓ Pass |
| SELECT TABLE_NAME FROM INFORMATION_SCHEMA.VIEWS | 5 views listed (VW_OPEN_ISSUES_PER_PROJECT, VW_DEVELOPER_WORKLOAD, VW_SPRINT_VELOCITY, VW_BUG_SEVERITY_BREAKDOWN, VW_RECENT_ACTIVITY) | ✓ Pass |
| SELECT name, OBJECT_NAME(parent_id) FROM sys.triggers | 5 triggers listed across ISSUE, COMMENT, PROJECT_MEMBER tables | ✓ Pass |
| SELECT i.name, t.name FROM sys.indexes i JOIN sys.tables t… | 10 non-clustered indexes listed | ✓ Pass |
| SELECT * FROM VW_OPEN_ISSUES_PER_PROJECT | 5 rows (one per project) with Critical/High/Medium/Low counts | ✓ Pass |
| SELECT * FROM VW_DEVELOPER_WORKLOAD | 8 developers with TotalAssigned, ActiveIssues, OverdueIssues, Resolved | ✓ Pass |
| SELECT * FROM VW_SPRINT_VELOCITY | 5 rows with VelocityPct (sprints 1-2 show partial completion) | ✓ Pass |
| SELECT * FROM VW_RECENT_ACTIVITY | ~25 rows of status changes from the simulated UPDATE block | ✓ Pass |
| SELECT COUNT(*) FROM ISSUE | 55 rows | ✓ Pass |
| SELECT COUNT(*) FROM STATUS_HISTORY | ~25 rows auto-generated by trigger | ✓ Pass |

Note: Replace this section with actual screenshots from your SSMS and browser demo video before submitting. Screenshots should show: (1) SSMS query results window for verification queries, (2) Browser showing the Dashboard page with activity feed, (3) Browser showing the Issue List with 55 rows, (4) Browser showing a successful bug creation form submission, (5) Browser showing a status update and its reflection in the audit trail.

### 9.2 Query Results Summary

All 15 analytical queries returned non-empty, meaningful results after seeding. Key findings:

- Q1 (Critical open bugs): 7 critical-priority bugs in Open status across 3 projects.
- Q2 (Top 5 developers): The simulate-activity block resolved issues 1–5 and 11–13, making Aleeza (UserID 1, admin) the top actor in STATUS_HISTORY. Among developers, Danyal (UserID 4) held the most resolved assignments.
- Q6 (Overloaded developers): With 55 issues distributed across 7 developers, several had >7 active assignments.
- Q10 (Issues with no comments): All 55 issues had zero comments in the seed data (no COMMENT inserts), providing a clean NOT EXISTS result set of all 55 rows.
- Q15 (Duplicate titles): Issues titled 'Issue Number X' are unique, so this query returned 0 rows, confirming the constraint works correctly.

### 9.3 Trigger Correctness Tests

| Trigger | Test Case | Expected Behaviour | Result |
|---|---|---|---|
| TR_Issue_StatusTransition | UPDATE ISSUE SET Status='In Review' WHERE IssueID=20 (currently Open) | ROLLBACK + RAISERROR: illegal transition | ✓ Blocked |
| TR_Issue_SubtypeEnforce | INSERT INTO ISSUE without matching BUG row (trigger enabled) | ROLLBACK + RAISERROR: subtype row missing | ✓ Blocked |
| TR_Issue_StatusAudit | UPDATE ISSUE SET Status='In Progress' WHERE IssueID=5 | Auto-insert into STATUS_HISTORY with correct ChangedBy | ✓ Row inserted |
| TR_Comment_AutoNumber | INSERT INTO COMMENT (IssueID, AuthorID, Content) — no CommentNo | Auto-assign CommentNo=1 for first comment on that issue | ✓ CommentNo=1 |
| TR_ProjectMember_MandatoryManager | DELETE FROM PROJECT_MEMBER WHERE ProjectID=1 AND UserID=2 (last manager) | ROLLBACK + RAISERROR: manager rule violated | ✓ Blocked |

## 10. Performance Metrics

### 10.1 Index Effectiveness

The 10 non-clustered indexes were designed using a query-pattern-first approach. The following analysis estimates their impact on the most critical operations:

| Query / Operation | Without Index | With Index | Index Used |
|---|---|---|---|
| All open issues in Project Alpha | Full table scan on ISSUE (55 rows) | Index seek on IX_Issue_Project_Status (ProjectID=1, Status≠Done/Closed) | IX_Issue_Project_Status |
| Developer workload view (VW_DEVELOPER_WORKLOAD) | Full scan of ISSUE_ASSIGNMENT then nested-loop join to ISSUE | Index seek on IX_IssueAssignment_UserID per developer | IX_IssueAssignment_UserID |
| Sprint velocity (VW_SPRINT_VELOCITY) | Full scan of SPRINT_ISSUE | Seek on IX_SprintIssue_SprintID | IX_SprintIssue_SprintID |
| Audit trail lookup for IssueID=5 | Full scan of STATUS_HISTORY | Seek + ordered read on IX_StatusHistory_Issue_Time | IX_StatusHistory_Issue_Time |
| Email-based user lookup | Full scan of USERS | Seek on IX_Users_Email (unique) | IX_Users_Email |

Note: At the 55-row scale of the seed data, full-table scans are fast regardless. The index design is validated by examining execution plans in SSMS (Query → Display Estimated Execution Plan), which shows Index Seek operations for all covered queries. The benefit becomes material at thousands of rows, representative of real-world deployment.

### 10.2 Trigger Overhead

Triggers introduce overhead because they execute within the same transaction as the originating DML statement. The five triggers in this system are designed to minimize this overhead:

- **TR_Issue_StatusAudit:** Fires only when Status actually changes (IF NOT UPDATE(Status) RETURN). At 55 rows with ~25 status changes, the overhead is negligible.
- **TR_Issue_StatusTransition:** Uses a single EXISTS check with a compact NOT-IN list of illegal transitions. No cursor or loop logic.
- **TR_Comment_AutoNumber:** INSTEAD OF trigger; replaces the INSERT with a single re-written INSERT using a MAX subquery. Overhead is one extra read per insert.
- **TR_ProjectMember_MandatoryManager:** Fires on DELETE and UPDATE but performs a fast NOT EXISTS correlated subquery.
- **TR_Issue_SubtypeEnforce:** Three EXISTS checks; each is a simple PK lookup (clustered seek) in BUG, FEATURE, TASK.

### 10.3 View Query Complexity

| View | Tables Joined | Aggregations | Estimated Cost (relative) |
|---|---|---|---|
| VW_OPEN_ISSUES_PER_PROJECT | PROJECT ⋈ ISSUE | GROUP BY ProjectID, 4× CASE SUM | Low; covered by IX_Issue_Project_Status |

| | | | |
|---|---|---|---|
| VW_DEVELOPER_WORKLOAD | USERS ⋈ ISSUE_ASSIGNMENT ⋈ ISSUE | GROUP BY UserID, 3× CASE SUM | Medium; three-table join; covered by IX_IssueAssignment_UserID |
| VW_SPRINT_VELOCITY | SPRINT ⋈ PROJECT ⋈ SPRINT_ISSUE ⋈ ISSUE | GROUP BY SprintID, 3× CASE SUM | Medium; four-table join; covered by IX_SprintIssue_SprintID |
| VW_BUG_SEVERITY_BREAKDOWN | BUG ⋈ ISSUE ⋈ PROJECT | GROUP BY ProjectID, Severity | Low; BUG is small (≈18 rows) |
| VW_RECENT_ACTIVITY | STATUS_HISTORY ⋈ ISSUE ⋈ PROJECT ⋈ USERS + WHERE 30-day filter | None (row-level) | Low with IX_StatusHistory_Issue_Time |

# 11. Reflection and Discussion

## 11.1 Assumptions Made

- The system operates for a single organisation (no multi-tenancy). All users share one BugTrackerDB.
- Authentication is simulated: the logged-in UserID is supplied by the application layer (via session_context) rather than validated at the database level.
- Issue IDs are application-managed integers (not IDENTITY columns) to simplify the transactional insert pattern in the subtype trigger.
- The 55 seeded issues use generated titles ('Issue Number X'); real data would contain descriptive titles.
- Status transition re-open (Done/Closed → Open) is intentionally allowed to represent the real-world 'reopen a fixed bug' workflow.

## 11.2 Challenges Encountered

- **Subtype enforcement trigger:** The TR_Issue_SubtypeEnforce trigger must evaluate after both the ISSUE and subtype rows have been inserted. In SQL Server, AFTER INSERT triggers fire at statement-end, not transaction-end, which means both rows must be visible within the same batch. This required using a T-SQL BEGIN TRY block in the Python endpoint to execute both INSERTs as a single batch string, rather than two separate cursor.execute() calls.
- **Session context for audit trigger:** TR_Issue_StatusAudit requires the acting user's ID to be injected via EXEC sys.sp_set_session_context before the UPDATE. This coupling between the application layer and the database trigger was a non-obvious requirement that required careful documentation to prevent Shaheer's backend from defaulting silently to the reporter ID.
- **pyodbc batch limitation:** The test_batch.py script revealed that pyodbc does not support multi-statement batches using semicolon separation in a single cursor.execute() call by default. The workaround used in the production code (a T-SQL BEGIN TRY…END CATCH block submitted as a single string) successfully executes both inserts atomically.

- **Normalisation of derived attribute:** ResolutionTime had to be implemented as a SQL Server computed column rather than a stored attribute to satisfy 3NF (a stored ResolutionTime would create a functional dependency on CreatedAt and the current time, a non-key transitive dependency). This decision required explicit documentation in the schema to avoid confusion.

### 11.3 Limitations of Current Design

- IssueID is manually assigned by the application; a production system would use SEQUENCE or IDENTITY to prevent collisions under concurrent inserts.
- There is no role-based access control at the database layer. Any user with database access can execute any query.
- The HTML frontend is minimal; it lacks client-side validation, pagination for the issue list, and real-time updates.
- The seed data uses generated titles, which means duplicate-detection queries (Q15) trivially return no results. Real data would stress-test this constraint more meaningfully.
- The system does not model file storage; ATTACHMENT.URL stores a link rather than file content.

### 11.4 Potential Improvements for Real-World Deployment

- Replace manual IssueID assignment with SEQUENCE objects to support concurrent multi-user inserts safely.
- Implement row-level security (RLS) in SQL Server so developers can only see issues assigned to their projects.
- Add full-text indexing on ISSUE.Title and COMMENT.Content to support keyword search.
- Introduce a notification table and background worker that polls STATUS_HISTORY and sends email alerts on assignment or status change.
- Partition the STATUS_HISTORY table by year for archival performance as the audit log grows.
- Migrate the frontend to a React or Vue SPA consuming the existing FastAPI endpoints, providing a smoother user experience with pagination, real-time updates via WebSockets, and client-side filtering.

## 12. References


1. Atlassian. (2002). Jira Software. Atlassian Corporation. https://www.atlassian.com/software/jira
2. Chen, P. P.-S. (1976). The entity-relationship model — toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9–36.
3. Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.
4. Date, C. J. (2003). An introduction to database systems (8th ed.). Addison-Wesley.

5. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.
6. GitHub. (2011). GitHub Issues. GitHub Inc. https://github.com/features/issues
7. Hoffer, J. A., Prescott, M. B., & McFadden, F. R. (2007). *Modern Database Management*. Prentice Hall.
8. Kerzner, H. (2017). Project management: A systems approach to planning, scheduling, and controlling (12th ed.). Wiley.
9. Microsoft. (2024). SQL Server documentation: Indexes, triggers, and computed columns. Microsoft Corporation. https://docs.microsoft.com/en-us/sql/
10. Ramakrishnan, R., & Gehrke, J. (2003). Database management systems (3rd ed.). McGraw-Hill.
11. Ramez Elmasri, & Sham Navathe. (2014). *Fundamentals of database systems*. Pearson.
12. Ramírez, S. (2018). FastAPI framework, high performance, easy to learn, fast to code, ready for production. https://fastapi.tiangolo.com
13. Smith, J. M., & Smith, D. C. P. (1977). Database abstractions: Aggregation and generalization. ACM Transactions on Database Systems, 2(2), 105–133.
14. Teorey, T. J., Yang, D., & Fry, J. P. (1986). A logical design methodology for relational databases using the extended entity-relationship model. ACM Computing Surveys, 18(2), 197–222.